
Topic 4 – Async Research Assistant

Topic Four

Nural Shukurlu • Emil Mammadov • Mahammadali Babayev • Bailar Bayramov

Final Project Defense – September 2026

The problem in one sentence

What the system does

You ask a research question in plain language; we query Wikipedia, arXiv, and web search at the same time and return one short answer with numbered citations.

Inputs

- ▶ One question string (3–2000 chars)
- ▶ Optional source subset, cache flag

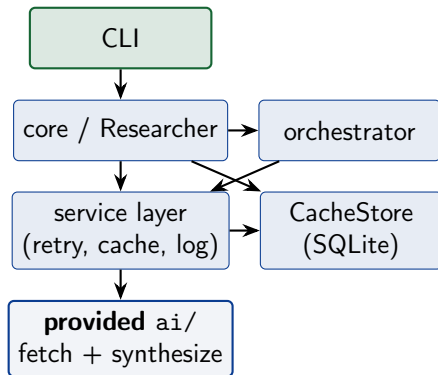
Outputs

- ▶ Answer text with [N] citations
- ▶ Reference list: title + URL + origin

Entry points: ask, demo, bench under `python -m researcher` (plus `-offline` for keyless runs)

Architecture

Module map. Only the service layer touches the *provided ai/* package.



Swapping Gemini for OpenAI touches one file plus .env. Pydantic models everywhere; no plain dicts cross modules.

Three design decisions we can defend

1. **asyncio over threads**

Three independent network waits; threads add risk for no gain, processes would be absurd for HTTP.

2. **Cache behind an ABC, service built by composition**

SQLite swaps for files without touching callers; the service *uses* a store instead of inheriting one.

3. **SQLite file over a database server**

Zero infrastructure, reproducible grading, honest trade: single-writer only.

Concurrency: the benchmark

Workload: 5 questions $\times$ 3 sources (15 fetches, 300 ms simulated each), cleared cache,
scripts/bench.py -offline

	Sequential	Concurrent (sem=3)
Wall-clock (s)	4.52	2.73
Speedup	1.0 $\times$	1.66 $\times$
Provider 429s	0	0

Bottleneck named: our own token bucket plus the single serial synthesis call

What didn't reach $N\times$: Sleeps dominate equally in both modes; the bucket caps bursts by design

Robustness: the stale-cache incident

Scenario: a demo answered “photosynthesis” with physics papers and zero Wikipedia hits

How we surfaced it: cache-hit logs showing zero items; empty hits cached with a 24h TTL kept re-serving

What the system does now:

- ▶ Retry: 3 attempts, exponential backoff 1–8 s plus jitter (transient errors only)
- ▶ Shape: keywords for Wikipedia/arXiv, full sentences for web search
- ▶ Cache: never store empties; versioned keys orphan old logic
- ▶ Degrade: one dead source becomes a warning, never a crash

Live demo (or scripted demo)

What we will run

One unseen question across all three live sources, then the scripted five-question digest.

Demo command (works inside Docker):

```
$ docker run --env-file .env finalproj \  
python -m researcher ask "Who was Marie Curie?"
```

- ▶ Watch the [N] citations resolve to titles and URLs on screen.
- ▶ `demo -limit 5` writes `artefacts/answers.json` plus a dated digest.

Fresh artefacts from a 5/5 live run ship in the repo. Offline mode needs no keys at all.

Testing

Coverage & shape

- ▶ Total coverage: 86 %
- ▶ Provided ai/ smoke tests: **passing** (16/16)
- ▶ Test count: 57 own + 16 smoke
- ▶ All tests offline; AI module fully mocked

Three tests worth calling out

- ▶ Happy-path end-to-end ask
- ▶ Concurrency: one source fails, answer still lands
- ▶ Error path: empty hits are never cached

Tooling

`pytest + pytest-asyncio + pytest-cov + respx / unittest.mock. mypy clean`
on 15 files, `ruff` clean.

Limitations & what we'd do next

Where the system is brittle today

- ▶ No provider failover; if Gemini is down, synthesis stops with a clean error.
- ▶ Single-process SQLite and rate limiter; scale needs Postgres plus a shared budget.
- ▶ Web search is the fragile leg; DuckDuckGo throttles some networks, Tavily is the paid escape.

Given another week we would

1. Add LLM failover with a chaos test killing the primary.
2. Load-test past the benchmark harness to measure the real ceiling.
3. Rank sources by quality so weak hits never get cited.

Team contributions

Member	Primary ownership
Nural Shukurlu	models, resilient service, rate limiter, config, touch-ups
Emil Mammadov	cache store (SQLite), parallel orchestrator
Mahammadali Babayev	core pipeline, demo/bench scripts, artefacts
Bailar Bayramov	CLI, Dockerfile, CI, README, report/slides assembly

AI tool use: coding assistance for routine work; the team reviewed, tested, and owns everything.

Questions?

Topic Four

`github.com/nuralnetworks/async-research-assistant`